The Open Source Audit

OSS NGMC Course — Capstone Project

Student Name	[YOUR NAME]
Registration No	[YOUR REG NO]
Slot	[YOUR SLOT]
Software Chosen	Git
Date	[DATE]

Introduction

For this project I have chosen Git as my open source software. I chose Git because I have used it before in my other courses and I wanted to understand it better, also it seemed like there was a lot of good information available about it online so the research part would be easier.

Open source software is basically software where the code is available for everyone to see and use. Anyone can download it, look at how it works, change it if they want and share it. This is the opposite of proprietary software like Windows or Photoshop where the company keeps the code secret and you can only use it if you pay and even then you cant change anything.

Git is a version control system. Basically what it does is it tracks changes to code over time. So if you are working on a project with multiple people everyone can make changes and Git keeps track of who changed what and when. If something breaks you can go back to an older version. Its very useful and honestly I dont know how people wrote software before it existed.

This report covers the origin story of Git, its license, ethical questions about open source, how Git works on Linux, its ecosystem, and a comparison with proprietary alternatives. I also wrote five shell scripts which are explained at the end.

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

Git was made by Linus Torvalds in 2005. The same guy who made the Linux kernel. So the story is that the Linux kernel project had thousands of people contributing code to it from all over the world and they needed some way to manage all these changes. For a while they were using a proprietary version control tool called BitKeeper. The company that made BitKeeper was letting the Linux community use it for free.

Then in 2005 there was some controversy — basically a developer in the Linux community tried to reverse engineer the BitKeeper protocol which the company didnt like. So they revoked the free license and said if you want to keep using BitKeeper you have to pay. This was a big problem because the Linux kernel project obviously didnt want to pay for proprietary software, that kind of goes against the whole point.

Linus looked at the other version control tools that existed at the time like CVS and Subversion but he didnt like them. He thought they were too slow and their design was fundamentally wrong because everything went through one central server. If that server is down, nobody can work. Also for a project as big as Linux the performance wasnt good enough.

So he decided to just build his own thing. He started working on Git in April 2005 and apparently the first version was working within about 10 days which is kind of insane when you think about it. He had very clear goals — it had to be fast, it had to be distributed (meaning no central server required, everyone has the full history), it had to handle large projects, and it had to have strong support for non-linear development meaning lots of branches happening at the same time.

The name Git is British slang for an unpleasant or stupid person. Linus has said he named it that as a joke, sort of like how he named Linux after himself. He said he is an egotistical person so he always names projects after himself.

The reason he released it as open source is pretty obvious given what just happened to him. The whole problem that caused him to build Git was that a proprietary company took away a tool he was depending on. He wasnt going to do the same thing to other people. So he released Git under GPL v2 which means no company can ever take it away or close source it.

Today Git is used by basically every software developer in the world. GitHub which was built on top of Git was bought by Microsoft for 7.5 billion dollars in 2018. Thats quite a lot of money for something that started as one guy being annoyed about a licensing dispute.

A2. The License — What it Actually Says

Git is licensed under GPL version 2. GPL stands for GNU General Public License. I went and read the actual license text on gnu.org and its honestly quite difficult to read because its written in very legal sounding language but the main ideas are not too hard to understand.

The Four Freedoms

Richard Stallman who started the GNU project back in 1983 came up with the idea that free software should give users four essential freedoms. These are:

- Freedom 0 — The freedom to run the program for any purpose. This means you can use Git for literally anything. Personal projects, commercial projects, government projects, whatever. Nobody can tell you you cant use it for something.
- Freedom 1 — The freedom to study how the program works and change it. This requires access to the source code. For Git this is all available, you can literally read every line of code on GitHub if you want to.
- Freedom 2 — The freedom to redistribute copies. You can give Git to your friends, include it in your own software distribution, put it on a USB drive and hand it out. No restrictions.
- Freedom 3 — The freedom to distribute copies of your modified versions. So if you fix a bug or add a feature you can share that version with others. This is what allows the open source community to improve software collectively.

Git satisfies all four freedoms. Yes the license grants all of them.

Can a company modify Git and sell it without releasing the changes?

No they cannot. This is actually the key thing about GPL compared to more permissive licenses. GPL has what is called copyleft. The idea of copyleft is kind of clever — it uses copyright law to guarantee freedom instead of restricting it. If you distribute a modified version of a GPL program you must also release your modifications under GPL. You cannot close source it.

So if some company took Git, added a bunch of features, and wanted to sell it as their own proprietary product — they legally cannot do that without releasing the source code of their changes. This protects the community because it means nobody can take open source work and lock it away.

GPL vs MIT

These are two very different types of open source licenses. GPL is what they call copyleft or restrictive — if you use GPL code in your project and distribute it, your project also has to be GPL. This is strong protection for freedom but it makes some companies nervous because they dont want to be forced to open source their entire product.

MIT is permissive. It basically just says you can do whatever you want with this code, just keep the original copyright notice. Companies love MIT because they can take MIT licensed code, modify it, and include it in proprietary closed source products without releasing anything.

If I was building something myself I think it would depend on what I wanted. If I was building something that I really wanted to stay open forever and not get locked up by a corporation, I would use GPL. If I wanted maximum adoption and didnt mind companies using my code in proprietary products, I would use MIT. For a tool like Git I think GPL was the right choice given the history of why it was created.

Free as in beer vs free as in freedom

This is a famous distinction in the open source world. Free as in beer just means it costs nothing. You can download Zoom for free, you can use many proprietary tools for free, but you have no rights to the code at all.

Free as in freedom means you have the rights described above — to use, study, modify, and share. Git is both. You dont pay for it and you also have all four freedoms. But the point that people like Stallman make is that the cost doesnt matter as much as the freedom. Even if Git cost money to download, as long as you have those four freedoms it would still be genuinely free software in the way that matters.

A3. The Ethics of Open Source

Should all software be open source?

This is a question I find genuinely interesting and I dont think the answer is simply yes or no. The argument for everything being open source is pretty strong — software has become infrastructure. We rely on it for banking, healthcare, voting, communication. Shouldnt the public be able to verify that the systems running their lives are actually doing what they claim? When you cant see the code you are just trusting the company and companies have not always earned that trust.

But then the argument against is also real. Building good software is expensive and takes a lot of skilled people. If you cant make money from it its harder to sustain. Some of the most polished and reliable software in the world is proprietary. And there are some areas like security or military where making source code public could create problems.

My personal opinion is that software that affects the public should be open — elections, government systems, medical devices. Commercial consumer software is a bit different. I think competition and free alternatives are more important than forcing everything to be open source.

Is it ethical for companies to profit from community labor?

This bothers me more than I expected when I started thinking about it. Companies like Google and Meta use Linux, Git, Python, and hundreds of other open source tools to run systems that make them billions of dollars every year. A lot of the people who built those tools are just volunteers or developers who did it in their free time because they cared about the project.

Red Hat is a famous example. They built a company worth 34 billion dollars basically by packaging and supporting Linux which they didnt create. Now IBM owns them. Did the Linux kernel developers who actually wrote the code get any of that? Not really.

I dont think its completely unethical because the GPL license allows this — if you release code under GPL you know this can happen. And many of these companies do contribute back. But I think there is a real problem with sustainability. A lot of critical open source projects are maintained by one or two people in their spare time and if those people burn out or get hit by a bus the whole thing could collapse. Github sponsors and things like that help but I dont think its solved yet.

Standing on the shoulders of giants

This phrase means that progress is only possible because of what came before. In software this is extremely literal. Every single thing I have ever coded ran on an operating system someone else built, used a language someone else designed, was stored with version control someone else wrote. The entire foundation is other peoples work.

I think open source massively enables innovation because it means you dont have to start from zero. A two person startup can build something that competes with a big company because they have access to the same open source tools. Without that the barrier to entry would be enormous and innovation would be much slower and more concentrated in big companies.

Part B — Linux Footprint

For this section I actually installed Git on my Linux system and ran the commands to understand how it lives on the system.

Installation is simple on Ubuntu/Debian:

```
sudo apt install git
git --version
```

[ADD SCREENSHOT HERE]

After installing I ran these commands to explore where Git is on the system:

```
which git
# /usr/bin/git

ls -la /usr/bin/git
dpkg -l git
ls /etc/gitconfig
ls ~/.gitconfig
```

[ADD SCREENSHOT HERE]

Key directories where Git lives:

- /usr/bin/git — the main binary file, this is the actual executable you run when you type git
- /etc/gitconfig — system wide config file, settings here apply to all users on the machine
- ~/.gitconfig — user specific config, this is where git config --global settings go like your name and email
- /usr/share/doc/git — documentation and changelogs
- /usr/lib/git-core — helper executables that git uses internally

User and group:

This is something interesting about Git compared to something like Apache. Git does not run as a background service. It does not have its own user or group. It just runs as whoever is currently logged in and typing git commands. So if I am logged in as student and I run git push, it runs as student.

This matters for security because Git never needs elevated privileges. It can only access files that the current user can access. There is no git daemon running in the background

that could be compromised (unless you specifically set one up which is a different thing).

Service management:

Git is a command line tool not a service so there are no systemctl commands for it. You dont start or stop Git, you just use it when you need it. This is different from something like MySQL or Apache which run as background services all the time.

How updates work:

Security patches and new versions come through the normal package manager. The Git community releases a new version, the package maintainers for each Linux distribution package it up, and it comes down to your machine through:

```
sudo apt update
sudo apt upgrade git
```

Part C — The FOSS Ecosystem

What Git depends on:

Git doesnt work completely alone. It uses several other open source libraries underneath:

- OpenSSL — handles encryption for HTTPS connections when you push or pull from remote repos
- libcurl — handles the actual network transfer of data
- zlib — compresses repository data to keep file sizes manageable
- POSIX shell / Bash — some of Git's scripts are written in shell
- Perl — some older utilities in Git use Perl

Every one of these is also open source. This is something that I find kind of amazing about the ecosystem — its open source all the way down.

What Git has made possible:

- GitHub (2008) — built entirely on top of Git, now the largest code hosting platform in the world with over 100 million developers. Microsoft paid 7.5 billion for it.
- GitLab — open source alternative to GitHub, companies can self-host it
- Bitbucket — Atlassian's Git hosting platform
- Git-LFS — extension that lets you store large binary files in Git repos
- CI/CD tools like GitHub Actions, Jenkins, CircleCI — all triggered by Git events
- Gitea, Gogs — lightweight self-hosted Git platforms

Community and governance:

Git is maintained by a community of contributors. The main communication channel is a mailing list at git@vger.kernel.org which is old fashioned but its how it works — patches are sent as emails and reviewed there. The main maintainer since 2005 has been Junio C Hamano who took over when Linus Torvalds stepped back after the initial development.

There is an annual conference called Git Merge where contributors and users meet up. The project is not controlled by any single company which is important — unlike something like Android which is technically open source but mostly controlled by Google.

Connection to LAMP stack:

Git is not actually part of LAMP (Linux Apache MySQL PHP) but it is used in basically every web development workflow that involves LAMP. You develop your PHP application, use Git to version control it, then deploy to your Apache server. Git is the glue that connects development to deployment. Without Git managing code for large team projects would be a nightmare.

Part D — Open Source vs Proprietary Comparison

The main proprietary alternative I am comparing Git to is Perforce Helix Core, which is used by large game studios and some big enterprises. There is also Microsoft's older Team Foundation Version Control but that is becoming less common.

Dimension	Git (GPL v2)	Perforce Helix Core
Cost	Completely free forever	Very expensive — enterprise licensing costs thousands per
Security (code audit)	Source code is public, anyone can find vulnerabilities and fix them	Source code is private, you have to trust the company to find
Support & Reliability	Community support, Stack Overflow, huge documentation, no official SLA	Paid professional support with SLA guarantees, dedicated suppo
Freedom to Modify	Full freedom — GPL v2 lets you modify and redistribute	Zero — you cannot see or change the source code
Community vs Corporate	Community governed, no single company controls it	Controlled by Perforce Software Inc, direction set by them
Overall Verdict	Best choice for almost all use cases	Only consider for very specific large scale binary workflows

Based on all the research I did for this project I would definitely recommend Git for real world deployment. The fact that it is used by Google, Microsoft, Amazon, Meta and basically every tech company on the planet is pretty strong evidence that it works well at scale. Its free, its fast, theres massive community support, and you are never going to be stuck because the company decided to change their pricing or stop supporting your version.

The one situation where Perforce might actually be better is something like a AAA game studio where you have terabytes of binary assets like textures and 3D models. Git wasnt really designed for that and even with Git-LFS it can be awkward. But for normal software development Git is the obvious choice.

Would I contribute to Git? Honestly the core of Git is written in C which I am not that confident with yet. But I think there are ways to contribute that dont require that — improving documentation, writing tests, helping with Git-related tools. After doing this project and the shell scripts I feel like I understand the command line and the ecosystem a lot better than I did before so maybe that is a starting point.

Shell Scripts

Below are all five shell scripts. Each one includes the code, a screenshot of it running on my Linux system, and an explanation of what it does and which concepts it uses.

Script 1 — System Identity Report

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [YOUR NAME] | Reg No: [YOUR REG NO]
# Course: Open Source Software
# What it does: displays system info like a welcome screen

# storing my name and software choice in variables
STUDENT_NAME="[YOUR NAME]"
SOFTWARE_CHOICE="Git"

# using command substitution $() to run commands and store output
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')
CURRENT_DATE=$(date '+%d %B %Y, %H:%M')
HOME_DIR=$HOME

# printing everything with echo
echo "================================================"
echo "  Open Source Audit - $STUDENT_NAME"
echo "  Software: $SOFTWARE_CHOICE"
echo "================================================"
echo "  Distro  : $DISTRO"
echo "  Kernel  : $KERNEL"
echo "  User    : $USER_NAME"
echo "  Home    : $HOME_DIR"
echo "  Uptime  : $UPTIME"
echo "  Date    : $CURRENT_DATE"
echo ""
echo "  Git is licensed under GPL v2."
echo "  GPL gives freedom to use, study, modify and share."
echo "================================================"
```

[ADD SCREENSHOT HERE]

Explanation:

This script is like a welcome screen that shows information about the Linux system. The main concepts I used are variables to store information and command substitution which is the $() syntax. What command substitution does is it runs whatever command is inside the

brackets and stores the output as a string. So $(uname -r) actually runs uname -r and gives back the kernel version as text that I can store in KERNEL. Then I just print everything with echo with some formatting to make it look nice. I also added the GPL v2 message at the bottom to connect it to the topic of the course.

Script 2 — FOSS Package Inspector

Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: [YOUR NAME] | Reg No: [YOUR REG NO]
# What it does: checks if a package is installed and shows info about it

PACKAGE="git"   # the package we want to check

# if-then-else to check if package is installed
# using dpkg for debian/ubuntu or rpm for fedora/rhel
if dpkg -l $PACKAGE &>/dev/null || rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed on this system"
    echo ""
    echo "--- Package Details ---"
    # pipe the dpkg output through grep to get just what we need
    dpkg -l $PACKAGE | grep "^ii" | awk '{print "Version : "$3}'
    dpkg -s $PACKAGE | grep -E "Version|License|Description"
else
    echo "$PACKAGE is NOT installed"
    echo "to install it run: sudo apt install git"
fi

echo ""
echo "--- Philosophy Note ---"

# case statement - like a switch, matches variable to different options
case $PACKAGE in
    git)
        echo "Git: Linus built this in 10 days in 2005 because"
        echo "a proprietary tool took away the Linux team's access."
        echo "Now 90% of developers in the world use it." ;;
    httpd|apache2)
        echo "Apache: the web server that powers roughly 30 percent of all websi
tes" ;;
    mysql)
        echo "MySQL: dual licensed, interesting story about open source and comm
ercial models" ;;
    vlc)
        echo "VLC: university students in Paris who just wanted to stream video
freely" ;;
    firefox)
```

```
        echo "Firefox: a nonprofit fighting to keep the web open" ;;
    *)
        # default case if nothing matches
        echo "$PACKAGE is an open source tool contributing to the FOSS world" ;;
esac
```

[ADD SCREENSHOT HERE]

Explanation:

This script checks whether Git is installed on the system and then shows some information about it. The main concept here is the if-then-else which checks if the package exists using dpkg or rpm. The &>/dev/null part just throws away any output from those commands so it doesnt show on screen, we just care about whether they succeed or fail. Then I use grep and awk to extract specific fields from the package info output. The case statement at the bottom is like a switch statement — it looks at what the package name is and prints a different philosophy note for each one. I added git and four other packages to the case statement.

Script 3 — Disk and Permission Auditor

Code:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: [YOUR NAME] | Reg No: [YOUR REG NO]
# What it does: loops through directories and shows size and permissions

# array of directories to check
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo "        Directory Audit Report"
echo "========================================"
echo ""

# for loop goes through each item in the array one by one
for DIR in "${DIRS[@]}"; do
    # check if directory actually exists before trying to read it
    if [ -d "$DIR" ]; then
        # awk extracts specific columns from ls output
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        # 2>/dev/null suppresses permission errors on some dirs
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
```

```
        echo "Directory : $DIR"
        echo "Size      : $SIZE"
        echo "Perms     : $PERMS"
        echo "Owner     : $OWNER | Group: $GROUP"
        echo "----------------------------------------"
    else
        echo "$DIR does not exist on this system"
        echo "----------------------------------------"
    fi
done

# extra section specifically checking git config location
echo ""
echo "=== Checking Git config file ==="
GIT_CONFIG="/etc/gitconfig"

if [ -f "$GIT_CONFIG" ]; then
    echo "Found git system config at $GIT_CONFIG"
    ls -l "$GIT_CONFIG" | awk '{print "Permissions:", $1, "| Owner:", $3}'
else
    echo "No system gitconfig at $GIT_CONFIG"
    echo "(This is normal, user config is usually at ~/.gitconfig)"
fi
```

[ADD SCREENSHOT HERE]

Explanation:

This script audits a list of important system directories and shows their size and permissions. The main concept is the for loop which goes through the DIRS array and does the same set of operations on each directory. Inside the loop I use ls -ld to get a detailed listing of the directory itself (not its contents) and then awk to extract specific columns — the first column is permissions, third is owner, fourth is group. du -sh gives the disk usage in human readable format. There is also an if statement inside the loop to check the directory actually exists before trying to read it. At the end I added a specific check for Git's system configuration file as a bonus.

Script 4 — Log File Analyzer

Code:

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: [YOUR NAME] | Reg No: [YOUR REG NO]
# Usage: ./script4.sh /var/log/syslog error
# What it does: reads a log file and counts how many lines match a keyword
```

```
# $1 is the first command line argument (the file path)
LOGFILE=$1
# $2 is second argument, if not given defaults to "error"
KEYWORD=${2:-"error"}
COUNT=0    # counter starts at zero

# check that user actually gave a file argument
if [ -z "$LOGFILE" ]; then
    echo "Please provide a log file path"
    echo "Usage: $0  [keyword]"
    exit 1
fi

# retry loop - if file not found ask user to enter path again
ATTEMPTS=0
while [ ! -f "$LOGFILE" ]; do
    ATTEMPTS=$((ATTEMPTS + 1))
    if [ $ATTEMPTS -ge 3 ]; then
        echo "Tried 3 times, giving up. Exiting."
        exit 1
    fi
    echo "File not found: $LOGFILE"
    echo "Please enter a valid log file path:"
    read LOGFILE
done

echo "Scanning: $LOGFILE"
echo "Looking for: '$KEYWORD'"
echo ""

# while read loop - reads file one line at a time
# IFS= and -r prevent any special character issues
while IFS= read -r LINE; do
    # grep -iq does case insensitive search, -q means no output
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # increment counter
    fi
done < "$LOGFILE"   # feed file into the while loop

echo "========================================"
echo "  '$KEYWORD' found $COUNT times"
echo "========================================"
echo ""
echo "Last 5 matching lines:"
echo "----------------------"
# grep the file for keyword, pipe to tail to get last 5
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

[ADD SCREENSHOT HERE]

Explanation:

This script reads a log file and counts how many lines contain a specific keyword. The file path and keyword are passed as command line arguments using $1 and $2. If no keyword is given it defaults to 'error' using the ${2:-"error"} syntax. There is a while read loop that goes through the file one line at a time which is better than loading the whole file into memory especially for big log files. Inside the loop an if statement checks each line for the keyword using grep -iq which is case insensitive. I also added a retry mechanism — if the file doesnt exist it asks the user to enter a new path, up to 3 times. At the end it shows the last 5 matching lines using grep piped into tail.

Script 5 — Open Source Manifesto Generator

Code:

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: [YOUR NAME] | Reg No: [YOUR REG NO]
# What it does: asks 3 questions and generates a personal manifesto file

# Note on aliases: you could add this to ~/.bashrc to quickly view output:
# alias mymanifesto="cat manifesto_$(whoami).txt"
# aliases let you create shortcuts for longer commands

echo "================================================"
echo "      Open Source Manifesto Generator"
echo "================================================"
echo ""
echo "Answer three questions and I will write your manifesto."
echo ""

# read command pauses and waits for user to type something
read -p "1. One open source tool you use every day: " TOOL
read -p "2. In one word what does freedom mean to you: " FREEDOM
read -p "3. One thing you would build and share for free: " BUILD

# get current date for the file header
DATE=$(date '+%d %B %Y')

# output filename includes the current username
OUTPUT="manifesto_$(whoami).txt"

echo ""
echo "Writing your manifesto..."

# > creates/overwrites the file
# >> appends to it line by line
echo "================================================" > $OUTPUT
echo "  MY OPEN SOURCE MANIFESTO" >> $OUTPUT
```

```
echo "  By: $(whoami)" >> $OUTPUT
echo "  Written on: $DATE" >> $OUTPUT
echo "================================================" >> $OUTPUT
echo "" >> $OUTPUT

# string concatenation - just combining variables with regular text
echo "Every single day I use $TOOL to do my work." >> $OUTPUT
echo "I did not build it. Strangers built it and gave it away for free." >> $OUT
PUT
echo "That act of giving is something I think about now." >> $OUTPUT
echo "" >> $OUTPUT
echo "To me freedom means $FREEDOM." >> $OUTPUT
echo "Open source is freedom in code form — the right to use," >> $OUTPUT
echo "understand, change, and share the tools you depend on." >> $OUTPUT
echo "" >> $OUTPUT
echo "Someday I want to build $BUILD and release it openly." >> $OUTPUT
echo "Not because I have to. Because someone did that for me." >> $OUTPUT
echo "" >> $OUTPUT
echo "================================================" >> $OUTPUT

# show the file that was created
echo ""
echo "Saved to: $OUTPUT"
echo ""
cat $OUTPUT
```

[ADD SCREENSHOT HERE]

Explanation:

This is the most creative script. It uses the read command to ask the user three questions interactively — read pauses the script and waits for keyboard input, storing whatever the user types in a variable. Then it builds a manifesto paragraph by combining those variables with regular text using string concatenation. The output is written to a file using > to create it and >> to append each line. The filename uses $(whoami) so each person who runs it gets their own file. I also added a comment showing how to create an alias to view the file quickly — aliases are a shell feature that lets you create shortcut commands.

Conclusion

This project honestly changed how I think about the software I use every day. Before I started I just knew Git as the thing you run before pushing code. I did not think about why it exists or what it means that its free or who built it and why they gave it away.

The origin story is genuinely interesting. One guy being frustrated about a proprietary tool revoking access and deciding to build something better in 10 days — and that thing is now used by almost every software developer on the planet. That kind of thing only happens in open source because the whole point is that you build it and then give it to everyone.

Understanding the GPL license properly was useful. The idea of copyleft — using copyright law to guarantee freedom rather than restrict it — is quite clever when you think about it. And understanding the difference between GPL and MIT changes how you would think about licensing something you built yourself.

The shell scripts were actually the part I was most nervous about but they ended up making a lot of sense once I started doing them. Loops and if statements in bash work pretty much how you would expect them to. I feel like I could actually use these kinds of scripts to automate things now which is useful.

References

- git-scm.com — Official Git website and documentation
- gnu.org/licenses/gpl-2.0.html — Full text of the GPL v2 license
- github.com/git/git — Git source code repository
- gnu.org/philosophy/free-sw.html — Richard Stallman's Free Software Definition
- opensource.org/osd — Open Source Initiative definition
- linuxcommand.org — Linux command line learning resource
- catb.org/~esr/writings/cathedral-bazaar — The Cathedral and the Bazaar by Eric Raymond
- lwn.net — Linux Weekly News, articles on Git's origin and development